GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 3

Implementación de control de versiones y automatización

Enunciado de la Actividad 3 · Maestría en Ingeniería de Software, Universidad de La Sabana

El encargo

> «La estrategia de gestión de configuración está aprobada. Ahora queremos verla funcionando: cada cambio revisado y probado automáticamente, y cada versión identificada y entregable con un solo paso.»
>
> Responsable del sistema que eligió cada grupo.

El sistema es el mismo de las unidades anteriores. Lo que se implementa es la estrategia de la Actividad 2: las ramas, las versiones, las líneas base y el control de cambios del F-09 al F-11. Lo que se construya aquí es la red de seguridad para las mejoras de la Unidad 4.

Actividad 3 · Implementación de control de versiones y automatización (DevOps)

Modalidad	Grupal, de 2 a 3 estudiantes · calificable, **30 %**
Dedicación	24 horas
Entregables	**1.** El repositorio del sistema en GitHub o GitLab. **2.** Un pipeline de CI/CD funcional: construcción, pruebas y despliegue. **3.** Un documento técnico de **5 a 7 páginas** o un **README.md**. **4.** Evidencias: capturas, registros de ejecución y commits.
Reto	¿Cómo implementar una estrategia de control de versiones y automatización que permita gestionar eficientemente la evolución del software?
Instrumento	Rúbrica, publicada en el aula virtual

Qué debe mostrar la entrega

#	Elemento	Qué debe mostrar	Formato
1	Repositorio	Estructura, convenciones de commits, rama principal protegida y roles escritos en el repositorio	F-15
2	Estrategia de ramas	La estrategia elegida, por qué es la adecuada para este sistema y este equipo, y cómo se aplicó	F-16
3	Versionamiento semántico	Las reglas de versión, las etiquetas, el registro de cambios y al menos una versión publicada	F-17

#	Elemento	Qué debe mostrar	Formato
4	Pipeline de CI/CD	Las etapas de construcción, pruebas y despliegue, sus disparadores, los artefactos y la aprobación	F-18
5	Evidencias	Ejecuciones reales del pipeline, una fallida y otra corregida, PR revisados y la versión entregada	F-19
6	Métricas de entrega	Frecuencia de entrega y tiempo de entrega de los cambios en el período del curso	F-20

Criterios de evaluación

La rúbrica tiene cinco criterios. Los niveles y los puntajes están en el aula virtual.

Criterio	Qué se observa en el nivel más alto
Implementación en Git	Uso correcto, estructurado y profesional de Git
Estrategia de ramas y versionamiento	Estrategia clara, consistente y bien aplicada
Automatización CI/CD	Pipeline funcional, completo y automatizado
Trazabilidad y control de cambios	Trazabilidad completa y verificable
Documentación y evidencias	Evidencias claras, completas y bien documentadas

Paso a paso

1. **Preparar el repositorio** (F-15): copiar la carpeta `plantilla/` de este repositorio a la raíz del repositorio del sistema y reemplazar el servicio de ejemplo por el sistema, o ajustar los comandos de prueba y construcción a su lenguaje.
2. **Proteger la rama principal**: exigir PR y que la CI pase antes de integrar.
3. **Definir y aplicar la estrategia de ramas** (F-16) y las **reglas de versión** (F-17).
4. **Hacer que el pipeline signifique algo** (F-18): pruebas que fallen cuando el sistema falla.
5. **Publicar al menos una versión** con una etiqueta y dejar que el pipeline de entrega la lleve hasta el final.
6. **Registrar las evidencias** mientras se trabaja (F-19), no al final.
7. **Medir** con `herramientas/entrega.py` (F-20), **verificar** con el F-21 y entregar.

```
git switch -c feat/primera-mejora
git commit -m "feat(api): agrega la ruta /estado"
git push -u origin feat/primera-mejora          # abrir el PR y ver correr la CI
git tag -a v0.2.0 -m "Versión 0.2.0" && git push origin v0.2.0
python herramientas/entrega.py <sistema>
```

Material de la unidad

- Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Capítulos 3 (ramas) y 5 (Git distribuido). https://git-scm.com/book
- Humble, J. (2015). *Continuous delivery* [Grabación de video]. Addison-Wesley Professional. Capítulos 1 a 3. Disponible en la biblioteca.
- GitLab. (2024). *CI/CD documentation*. https://docs.gitlab.com/ee/ci/
- Complementarios: Docker, *Docker documentation*, https://docs.docker.com; ThoughtWorks, *Technology Radar*, https://www.thoughtworks.com/radar

Errores frecuentes

- **Un pipeline que nunca falla.** Si las pruebas no pueden fallar, el pipeline no protege nada. Muestren una ejecución fallida y su corrección.
- **Integrar directo a main.** Si main no está protegida, la estrategia de ramas es solo una intención.
- **Construir en cada etapa.** Lo que se prueba en staging debe ser exactamente lo que llega a producción.
- **Etiquetas que no coinciden con la versión declarada.** El script `verificar_version.py` de la plantilla lo impide.
- **Evidencias reconstruidas al final.** Se nota, y suelen faltar las ejecuciones fallidas.
- **Una estrategia distinta de la diseñada en la Unidad 2** sin explicar por qué cambió.
